

Installation instructions

Install ROS2 Humble

Ubuntu (Debian packages)

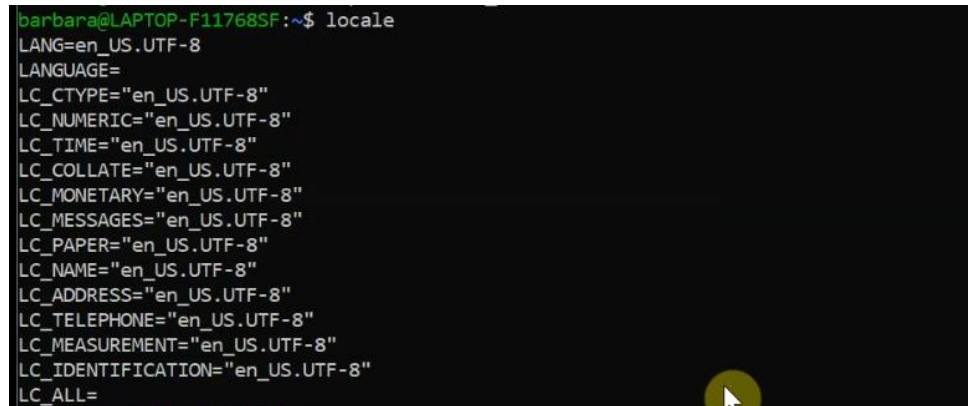
Set locale

Write in the Linux terminal:

```
locale # check for UTF-8

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # verify settings
```

A terminal window screenshot showing the output of the 'locale' command. The output lists various locale settings, all of which are set to 'en_US.UTF-8'. The terminal has a dark background with green text. The prompt is 'barbara@LAPTOP-F11768SF:~\$'.

```
barbara@LAPTOP-F11768SF:~$ locale
LANG=en_US.UTF-8
LANGUAGE=
LC_CTYPE="en_US.UTF-8"
LC_NUMERIC="en_US.UTF-8"
LC_TIME="en_US.UTF-8"
LC_COLLATE="en_US.UTF-8"
LC_MONETARY="en_US.UTF-8"
LC_MESSAGES="en_US.UTF-8"
LC_PAPER="en_US.UTF-8"
LC_NAME="en_US.UTF-8"
LC_ADDRESS="en_US.UTF-8"
LC_TELEPHONE="en_US.UTF-8"
LC_MEASUREMENT="en_US.UTF-8"
LC_IDENTIFICATION="en_US.UTF-8"
LC_ALL=
```

Setup Sources

Write in the Linux terminal:

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

```
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-
archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee
/etc/apt/sources.list.d/ros2.list > /dev/null
```

Install ROS 2 packages

Write in the Linux terminal:

```
sudo apt update  
sudo apt upgrade
```

```
sudo apt install ros-humble-desktop
```

```
sudo apt install ros-humble-ros-base
```

```
sudo apt install ros-dev-tools
```

Environment setup

Write at the end of the .bashrc:

```
source /opt/ros/humble/setup.bash
```

```
$ .bashrc  X  
Ubuntu-22.04 > home > barbara > $ .bashrc  
117 fi  
118  
119 source /opt/ros/humble/setup.bash
```

Close and open again the Linux terminal.

Check environment variables

Write in the Linux terminal and check that: ROS_VERSION=2 ROS_PYTHON_VERSION=3 ROS_DISTRO=humble

```
printenv | grep -i ROS
```

```
barbara@LAPTOP-F11768SF:~$ printenv | grep -i ROS  
ROS_VERSION=2  
ROS_PYTHON_VERSION=3  
AMENT_PREFIX_PATH=/opt/ros/humble  
PYTHONPATH=/opt/ros/humble/lib/python3.10/site-packages:/opt/ros/humble/local/lib/python3.10/dist-packages  
LD_LIBRARY_PATH=/opt/ros/humble/opt/rviz_ogre_vendor/lib:/opt/ros/humble/lib/x86_64-linux-gnu:/opt/ros/humble/lib  
ROS_LOCALHOST_ONLY=0  
PATH=/opt/ros/humble/bin:/home/barbara/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/CanonicalGroupLimited.Ubuntu22.04LTS_2204.3.63.0_x64__79rhkp1fndgsc:/mnt/c/Users/UX431/AppData/Local/Programs/Python/Python38:/mnt/c/Python312/Scripts:/mnt/c/Python312:/mnt/c/Program Files/Semeru/jre-11.0.13.8-openj9/bin:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/MiKTeX/miktex/bin/x64:/mnt/c/Program Files/MATLAB/R2021b/runtime/win64:/mnt/c/Program Files/MATLAB/R2021b/bin:/mnt/c/Program Files (x86)/Windows Kits/8.1/Windows Performance Toolkit:/mnt/c/Program Files/Calibre2:/mnt/c/Program Files/Docker/Docker/resource s/bin:/mnt/c/ProgramData/DockerDesktop/version-bin:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/ProgramData/chocolatey/lib/cunit/lib:/mnt/c/ProgramData/chocolatey/lib/tinym2/lib:/mnt/c/ProgramData/chocolatey/lib/bullet/lib:/mnt/c/Program Files/OpenSSL-Win64/bin:/mnt/c/Program Files/CMake/bin:/mnt/c/opencv/x64/vc16/bin:/mnt/c/xmlint/bin:/mnt/c/Program Files/Graphviz/bin:/mnt/c/Program Files/Git/cmd:/mnt/c/Users/UX431/AppData/Local/Programs/Python/Python38/Scripts:/mnt/c/Users/UX431/AppData/Local/Programs/Python/Python38:/mnt/c/Users/UX431/AppData/Local/Programs/Python/Python310/Scripts:/mnt/c/Users/UX431/AppData/Local/Programs/Python/Python310:/mnt/c/Users/UX431/AppData/Local/Microsoft WindowsApps:/mnt/c/Users/UX431/AppData/Local/atom/bin:/mnt/c/Users/UX431/AppData/Local/Programs/Microsoft VS Code/bin:/snap/bin  
ROS_DISTRO=humble
```

Install SOFA

Building Sofa

Compiler

Write in the Linux terminal:

```
sudo apt install build-essential software-properties-common
```

GCC

Write in the Linux terminal:

```
apt-cache search '^gcc-[0-9.]+$'
```

Install the latest one (example with gcc-11):

```
sudo apt install gcc-11
```

Clang

Write in the Linux terminal:

```
apt-cache search '^clang-[0-9.]+$'
```

Install the latest one (example with clang-12):

```
sudo apt install clang-12
```

CMake: Makefile generator

Write in the Linux terminal:

```
sudo apt install cmake cmake-gui
```

Ninja: build system

Write in the Linux terminal:

```
sudo apt install ninja-build
```

CMake: caching system

Write in the Linux terminal:

```
sudo apt install ccache
```

Dependencies: Core (required)

tinyXML2

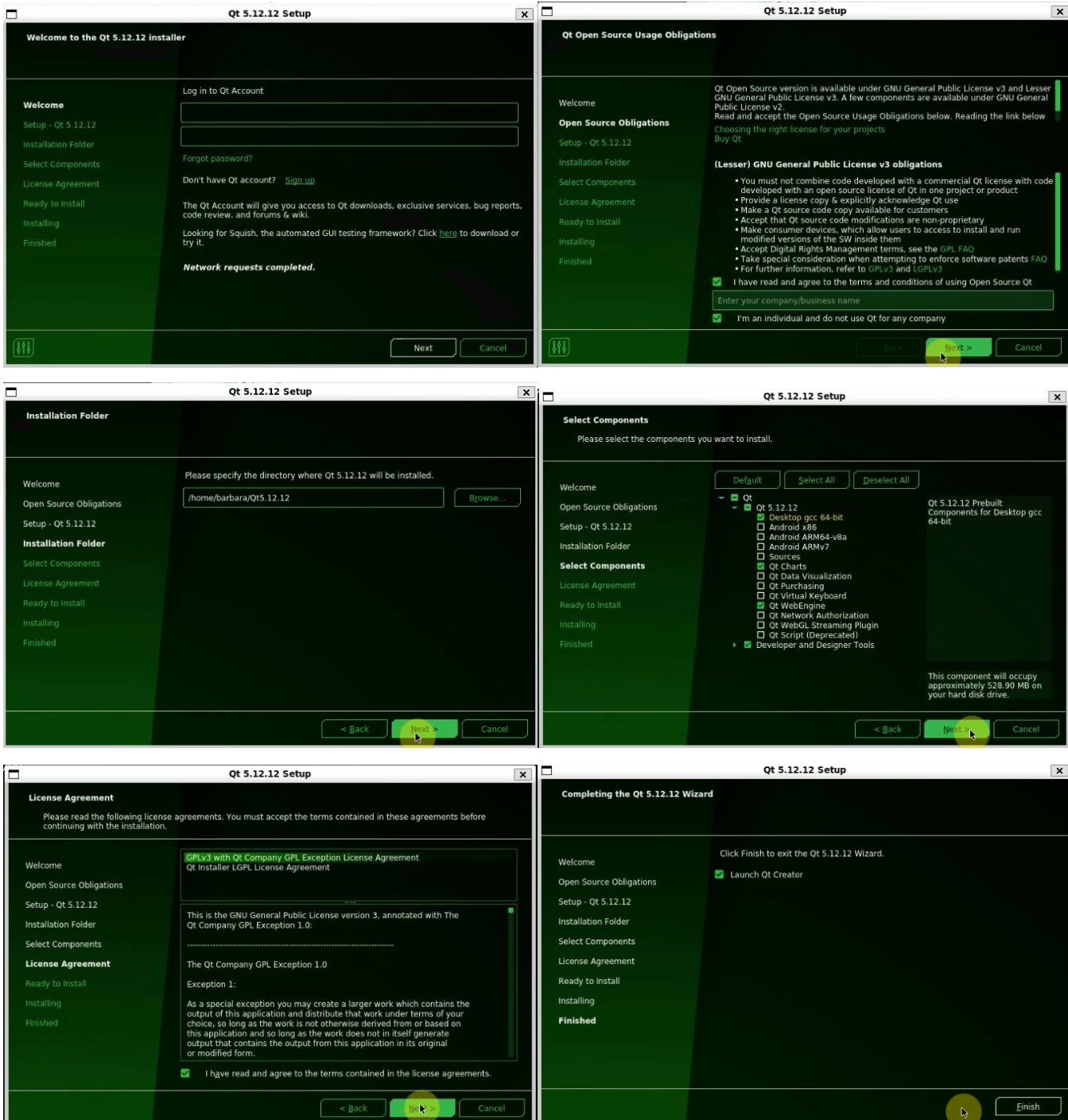
Write in the Linux terminal:

```
sudo apt install libtinyxml2-dev
```

Qt

Download Qt >= 5.12.0 with **Charts** and **WebEngine**. It is recommendable to install Qt **in your user directory** with the unified installer.

<https://download.qt.io/archive/qt/5.12/>



OpenGL

Write in the Linux terminal:

```
sudo apt install libopengl0
```

Boost

Write in the Linux terminal:

```
sudo apt install libboost-all-dev
```

Python 3.10 + pip + numpy + scipy

Write in the Linux terminal:

```
sudo apt install python3.10-dev pybind11-dev
sudo apt-get install python3.10-distutils
sudo curl -L https://bootstrap.pypa.io/pip/get-pip.py -output /tmp/get-pip3.py
sudo python3.10 /tmp/get-pip3.py
sudo python3.10 -m pip install --upgrade pip
sudo python3.10 -m pip install numpy scipy pybind11==2.9.1
```

Additional libraries: libPNG, libJPEG, libTIFF, Glew, Zlib

Write in the Linux terminal:

```
sudo apt install libpng-dev libjpeg-dev libtiff-dev libglew-dev zlib1g-dev
```

Eigen

Write in the Linux terminal:

```
sudo apt install libeigen3-dev
```

Dependencies: Plugins (optional)

CGALPlugin

Write in the Linux terminal:

```
sudo apt install libcgall-dev libcgall-qt5-dev
```

MeshSTEPLoader

Write in the Linux terminal:

```
sudo apt install liboce-ocaf-dev
```

SofaAssimp

Write in the Linux terminal:

```
sudo apt install libassimp-dev
```

SofaCUDA

Write in the Linux terminal:

```
sudo apt install nvidia-cuda-toolkit
```

SofaHeadlessRecorder

Write in the Linux terminal:

```
sudo apt install libavcodec-dev libavformat-dev libavutil-dev libswscale-dev
```

SofaPardisoSolver

Write in the Linux terminal:

```
sudo apt install libblas-dev liblapack-dev
```

Setup your source and build directories

Create SOFA directories as follows:

```
sofa_v23.12/  
├─ build/  
└─ src/  
    └─ < SOFA sources here >
```

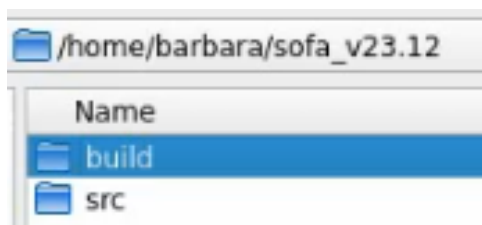
In the Linux terminal, go into sofa_v23.12/src/ folder and clone the last version (example for version v23.12):

```
git clone -b v23.12 https://github.com/sofa-framework/sofa.git
```

```
barbara@LAPTOP-F11768SF:~/sofa_v23.12$ git clone -b v23.12 https://github.com/sofa-f  
ramework/sofa.git src  
Cloning into 'src'...  
remote: Enumerating objects: 442286, done.  
remote: Counting objects: 100% (1363/1363), done.  
remote: Compressing objects: 100% (666/666), done.  
Receiving objects: 3% (13269/442286), 3.36 MiB | 6.70 MiB/s
```

Generate a Makefile with CMake

Create build directories respecting the arrangement



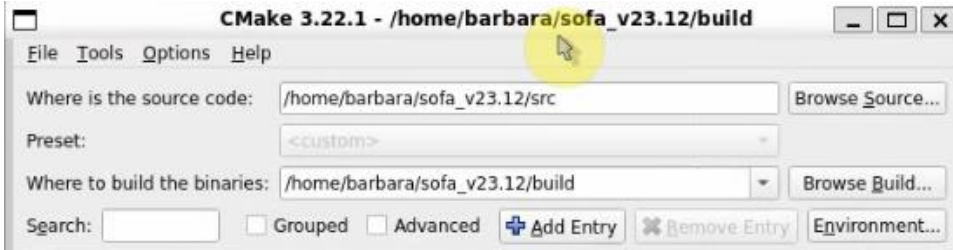
In Linux terminal and Run CMake-GUI

cmake-gui

```
barbara@LAPTOP-F11768SF:~/sofa_v23.12/build$ cmake-gui
```

Set source folder and build folder.

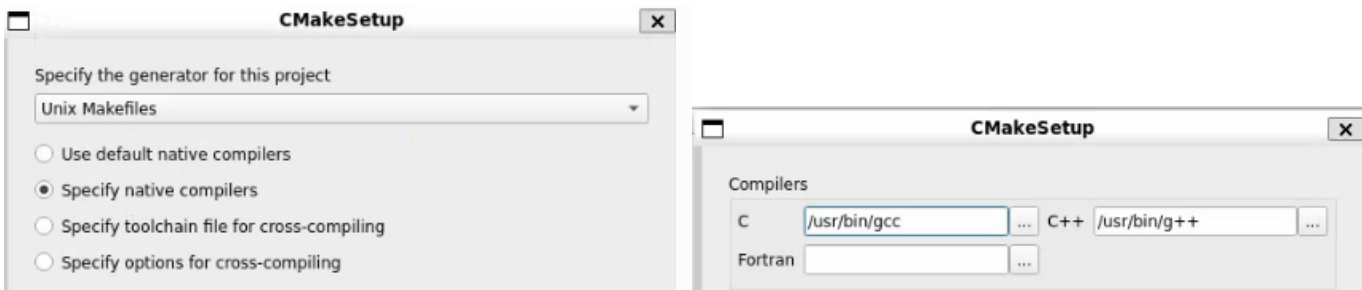
Run Configure.



Select “Unix Makefile”, “Specify native compilers” and press “Next”.

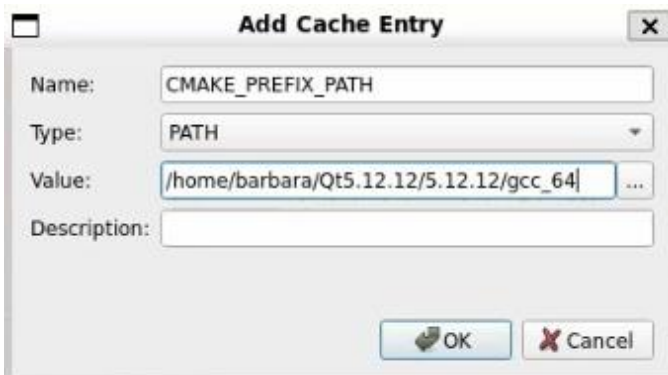
Set the C compiler to “/usr/bin/gcc” and C++ compiler to “/usr/bin/g++”.

Run Configure



Qt detection error

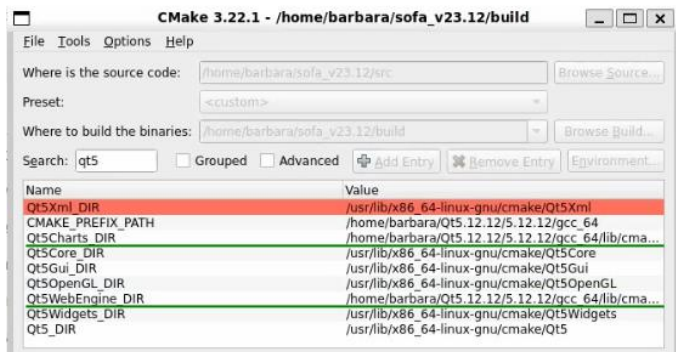
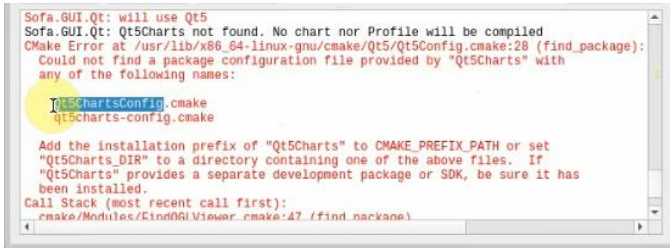
Add an entry called CMAKE_PREFIX_PATH with path /home/YOUR_USERNAME/Qt/QT_VERSION/COMPILER matching your Qt installation. Then, Configure again.



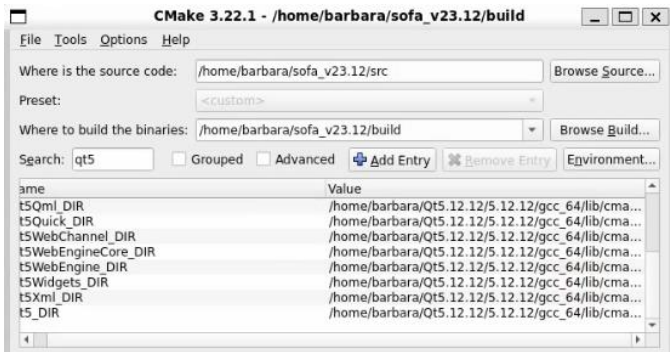
QtCharts and QtWebEngine error

On Qt5_Charts_DIR insert the path “/home/barbara/Qt5.12.12/5.12.12/gcc_64/lib/cmake/QtCharts”. Then, Configure again.

Do the same procedure with QtWebEngine.



Do the same with all Qt5XXX_DIR that is not directed to the correct file.

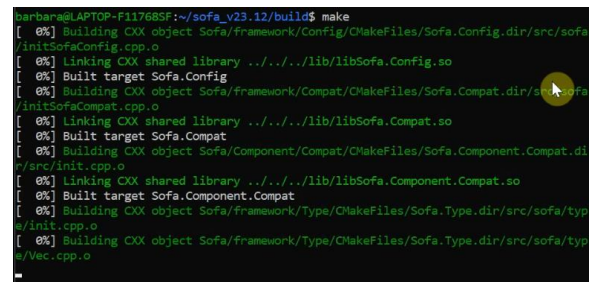


When you are ready, Configure again and run Generate

Compile

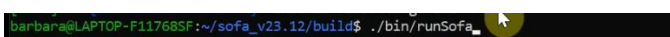
Open a terminal in your build directory and run

```
make
```



Then run Sofa

```
./bin/runSofa
```



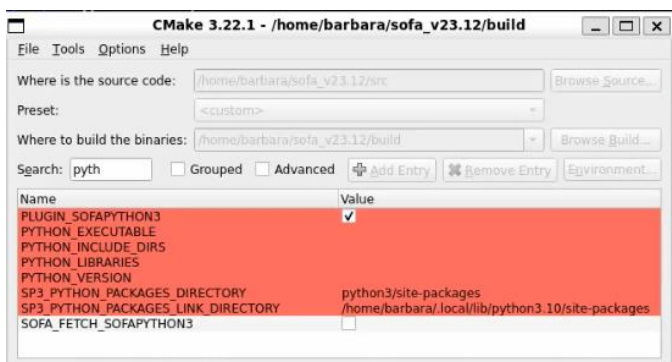
Pugins

Python (internal)

In CMake-GUI, check the box “SOFA_FETCH_SOFAPYTHON3”. Then, Configure.



The option “PLUGIN_SOFAPYTHON3” will appear together with other things. The “PLUGIN_SOFAPYTHON3” box should be checked and the “SOFA_FETCH_SOFAPYTHON3” should be unchecked. Then, Configure again.



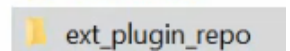
Finally, run Generate and on the terminal run “make” again.

```
make
```

ROS2 (external)

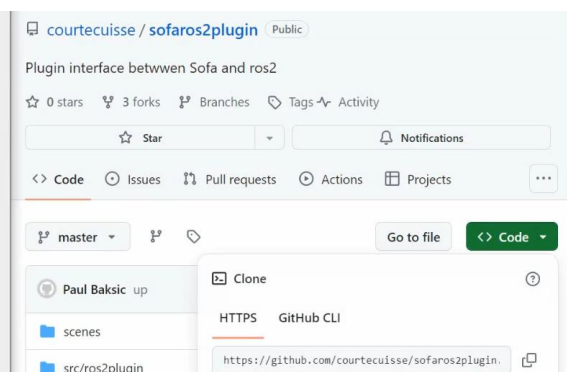
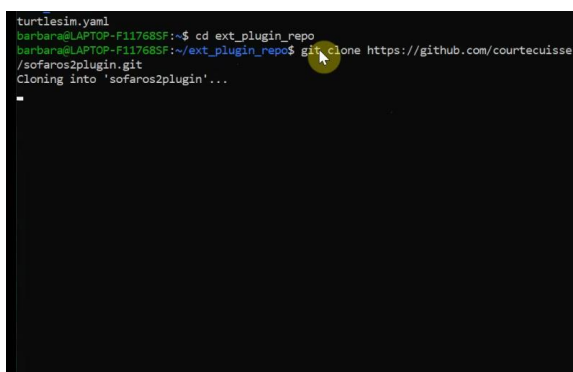
Create a folder called “ext_plugin_repo” outside of the “sofa” folder. In this directory, the structure is:

- ext_plugin_repo/
 - plugin1/
 - plugin2/
 - ...



Clone the GitHub sofarnos2plugin repository on the “ext_plugin_repo” folder.

<https://github.com/InfinyTech3D/sofarnos2plugin>

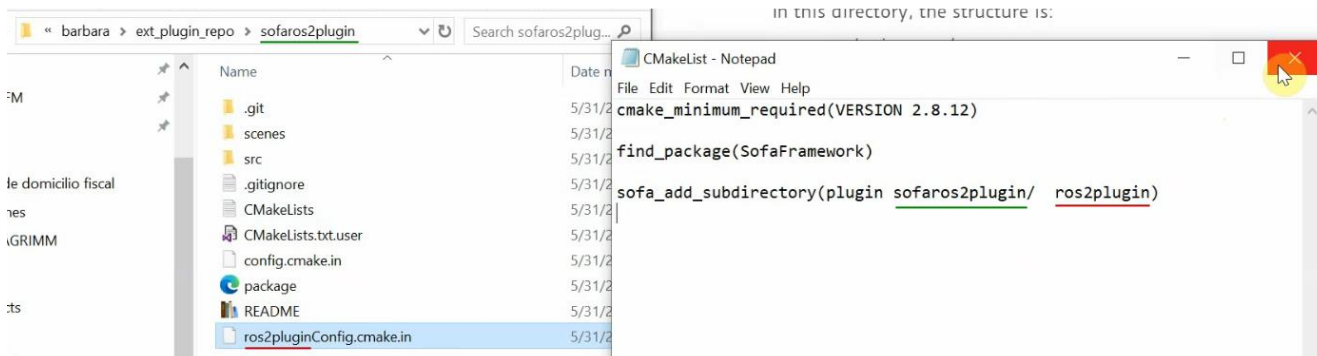


Create a CMakeList.txt and write on it as follows:

```
cmake_minimum_required(VERSION 2.8.12)
find_package(SofaFramework)
sofa_add_subdirectory(plugin path_to_plugin1/ name_of_project_plugin1)
sofa_add_subdirectory(plugin path_to_plugin2/ name_of_project_plugin2)
```



In the case of the ROS2 plugin, it will be:



To compile all the external plugins located in this repository, all you need to do is to set the path to this repository (/ext_plugin_repo/) in the CMake variable: **SOFA_EXTERNAL_DIRECTORIES**.



Then, Configure again. Finally, run Generate and on the terminal run “make” again.

```
make
```

.bashrc

Add to the .bashrc the following lines:

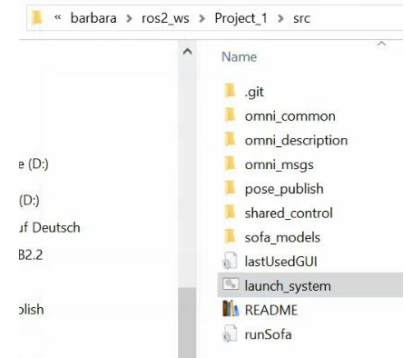
```
$ .bashrc
Ubuntu-22.04 > home > barbara > $ .bashrc
121
122 # SOFA
123 export PATH=$PATH:~/sofa_v23.12/build/bin
124 export SOFA_ROOT=~/sofa_v23.12/build
125 export SOFA_INSTALL_DIR=~/sofa_v23.12/build/lib
126 export PYTHONPATH=$PYTHONPATH:~/sofa_v23.12/build/lib/python3/site-packages
127 export SOFAPYTHON3_ROOT=~/sofa_v23.12/build/lib/python3/site-packages
```

Set up the project

Setup your source and build directories

Create directories as follows inside the ros2_ws and clone the project on the src folder.

```
ros2_ws/  
└─ Project_1/  
    └─ src/  
        └─ < clone project here >
```



Add to the .bashrc the following lines:

```
$ launch_system.sh  $ .bashrc  X  
Ubuntu-22.04 > home > barbara > $ .bashrc  
118  
119 # ROS2  
120 source /opt/ros/humble/setup.bash  
121 source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash  
122 source ~/ros2_ws/Project_1/install/setup.bash  
123
```

Change the .launch_system to customized it to your personal folder names:

```
$ launch_system.sh X  
Ubuntu-22.04 > home > barbara > ros2_ws > Project_1 > src > $ launch_system.sh  
1 # Source the ROS 2 installation  
2 source /opt/ros/humble/setup.bash  
3  
4 # Launch the first terminal and execute the first command  
5 gnome-terminal -- bash -c "source ~/ros2_ws/Touch_ROS2_SOFA2_V0/install/se  
6  
7 # Launch the second terminal and execute the second command  
8 gnome-terminal -- bash -c "source ~/ros2_ws/Touch_ROS2_SOFA2_V0/install/se  
9  
10 # Launch the third terminal and execute the third command  
11 gnome-terminal -- bash -c "source ~/ros2_ws/Touch_ROS2_SOFA2_V0/install/se  
12  
13 # Launch the fourth terminal and execute the third command  
14 gnome-terminal -- bash -c "source ~/ros2_ws/Touch_ROS2_SOFA2_V0/install/se  
15
```

On the terminal, on the project folder, run:

```
colcon build
```

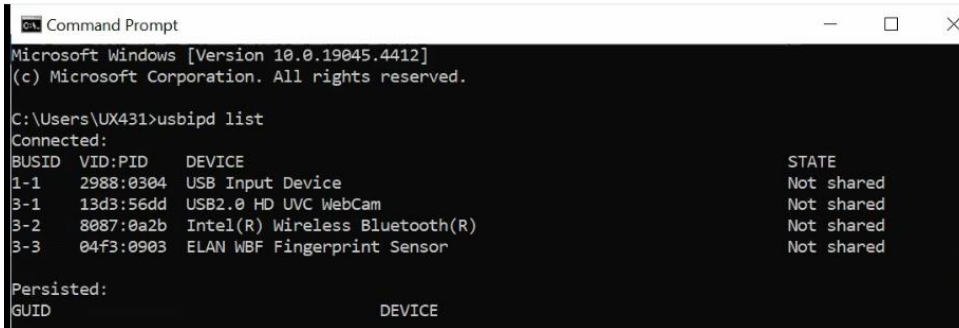
```
barbara@LAPTOP-F11768SF:~/ros2_ws/Project_1$ colcon build  
Starting >>> omni_msgs  
Starting >>> omni_description  
Starting >>> shared_control  
Finished <<< shared_control [1.90s]  
Finished <<< omni_description [4.62s]  
Finished <<< omni_msgs [15.6s]  
Starting >>> omni_common  
Starting >>> pose_publish  
[27.6s] [3/5 complete] [2 ongoing] [omni_common:cmake - 10.9s] ...
```

Installation of drivers

Only for WSL: Patch USB device from Win to WSL

On Windows command window run

```
usbipd list
```



```
Microsoft Windows [Version 10.0.19045.4412]
(c) Microsoft Corporation. All rights reserved.

C:\Users\UX431>usbipd list
Connected:
BUSID  VID:PID      DEVICE                      STATE
1-1    2988:0304  USB Input Device           Not shared
3-1    13d3:56dd  USB2.0 HD UVC WebCam       Not shared
3-2    8087:0a2b  Intel(R) Wireless Bluetooth(R) Not shared
3-3    04f3:0903  ELAN WBF Fingerprint Sensor Not shared

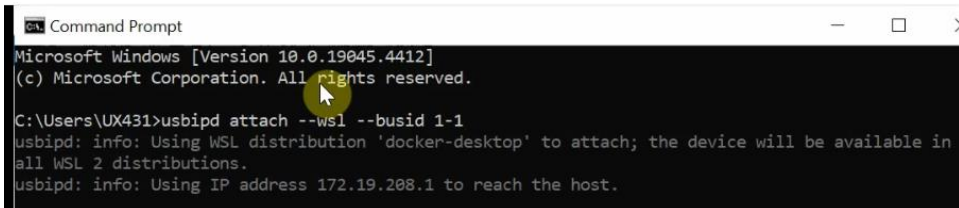
Persisted:
GUID                      DEVICE
```

Note which BUSID corresponds to "USB Input Device" (in the example 1-1). Then, as admin run the following command:

```
usbipd bind --busid 1-1
```

Next step has to be made every time that you reconnect the haptic device. It should be run in a no admin command window:

```
usbipd attach --wsl --busid 1-1
```



```
Microsoft Windows [Version 10.0.19045.4412]
(c) Microsoft Corporation. All rights reserved.

C:\Users\UX431>usbipd attach --wsl --busid 1-1
usbipd: info: Using WSL distribution 'docker-desktop' to attach; the device will be available in
all WSL 2 distributions.
usbipd: info: Using IP address 172.19.208.1 to reach the host.
```

Drivers installation in Linux

Add to the .bashrc:

```
export GTDD_HOME=/usr/share/3DSystems
```



```
$ .bashrc
Ubuntu-22.04 > home > barbara > $ .bashrc
128
129 # Omni drivers
130 export GTDD_HOME=/usr/share/3DSystems
```

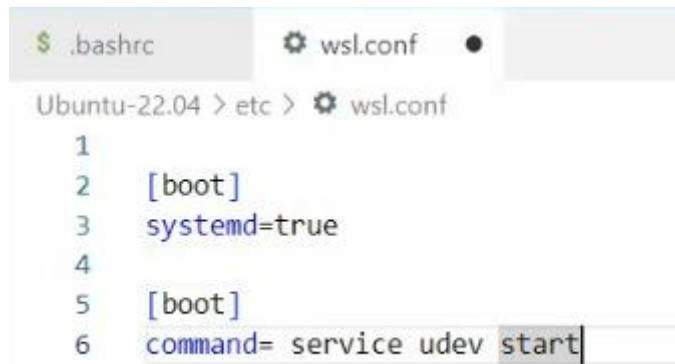
On the terminal:

```
sudo mkdir /usr/share/3DSystems/config -p
```

```
sudo chmod -R 777 /usr/share/3DSystems/
```


add to /etc/wsl.conf:

```
[boot] command= service udev start
```

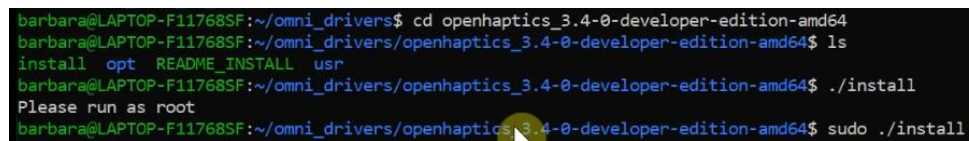


```
$ .bashrc wsl.conf
Ubuntu-22.04 > etc > wsl.conf
1
2 [boot]
3 systemd=true
4
5 [boot]
6 command= service udev start
```

On the terminal:

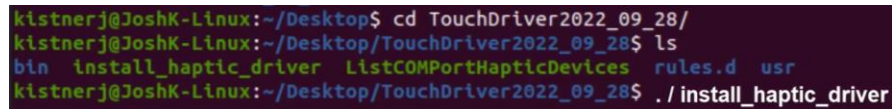
```
sudo usermod -a -G users [yourUSER]
```

Install openhaptics_3.4-0-developer-edition-amd64. To make this, go into the openhaptics_3.4-0-developer-edition-amd64 folder and execute the installation file as follows:



```
barbara@LAPTOP-F11768SF:~/omni_drivers$ cd openhaptics_3.4-0-developer-edition-amd64
barbara@LAPTOP-F11768SF:~/omni_drivers/openhaptics_3.4-0-developer-edition-amd64$ ls
install  opt  README  INSTALL  usr
barbara@LAPTOP-F11768SF:~/omni_drivers/openhaptics_3.4-0-developer-edition-amd64$ ./install
Please run as root
barbara@LAPTOP-F11768SF:~/omni_drivers/openhaptics_3.4-0-developer-edition-amd64$ sudo ./install
```

install TouchDriver_2023_01_12. To make this, go into the TouchDriver_2023_01_12 folder and execute the installation file as follows:



```
kistnerj@JoshK-Linux:~/Desktop$ cd TouchDriver2022_09_28/
kistnerj@JoshK-Linux:~/Desktop/TouchDriver2022_09_28$ ls
bin  install_haptic_driver  ListCOMPortHapticDevices  rules.d  usr
kistnerj@JoshK-Linux:~/Desktop/TouchDriver2022_09_28$ ./install_haptic_driver
```